

Especificaciones Generales

- **Modalidad:** El proyecto es grupal, desarrollado por equipos de máximo **04 integrantes**.
- **Propósito Central:** Diseñar e implementar desde cero un **Mini-Gestor de Bases de Datos Multimodal** que opere directamente sobre memoria secundaria (disco), estructurando páginas y bloques físicos binarios mediante punteros y serialización a bajo nivel (`seek`, `read`, `write`, `struct`).
- **Restricción Arquitectural :** Queda **terminantemente prohibido** utilizar motores de bases de datos existentes (SQLite, PostgreSQL, MySQL, DuckDB, etc.) o librerías ORM para gestionar el almacenamiento, las páginas o los índices. Toda estructura física debe ser codificada por los estudiantes desde cero.
- **Evaluaciones :**
 - **Entregable 1 (Proyecto 1 - P1: Semana 7):** Núcleo del motor relacional en disco (almacenamiento paginado, Heap File, Sequential File con overflow y reorganización, Árbol B^+ , Hashing Dinámico en disco, Parser SQL básico, backend API REST, cliente web con dashboard de I/O y experimentación masiva).
 - **Entregable 2 (Proyecto 2 - P2: Semana 16):** Extensión multimodal (Módulo Espacial R-Tree con visor de mapas, Módulo Textual con índice invertido SPIMI/BM25, Módulo Vectorial con búsqueda de similitud aproximada IVF/HNSW sobre embeddings) y despliegue de una **Aplicación de Inteligencia Artificial** completa (RAG, Recomendador, Detección de Copyright o Reconocimiento Facial).

1. Introducción y Motivación

En los sistemas gestores modernos de bases de datos, los motores ya no procesan exclusivamente registros tabulares tradicionales; conviven de forma integrada con geometrías espaciales, texto libre no estructurado y representaciones vectoriales densas generadas por modelos profundos de Inteligencia Artificial.

El objetivo de este proyecto es que el estudiante tenga un conocimiento profundo de cómo operan internamente los motores de bases de datos profesionales, implementando de forma progresiva cada

componente de la pila de software:

1. **Minimización del Costo de I/O:** El cuello de botella histórico y físico en sistemas de gestión de datos radica en la transferencia de bloques entre memoria secundaria y RAM. Cada diseño debe concebirse pensando en la reducción del número de accesos a disco.
2. **Estructuras de Indexación Avanzadas:** Implementación rigurosa de árboles multinivel balanceados y esquemas de dispersión dinámicos sobre archivos binarios en memoria secundaria.
3. **Procesamiento Declarativo de Consultas:** Construcción de un parser SQL con planificador de consultas, cálculo analítico de costos y telemetría de ejecución en tiempo real expuesta mediante una API REST y un cliente web interactivo.

2. Entregables de proyecto

La siguiente tabla resumen la división del proyecto entre ambos entregables :

Componente	Entregable 1: Motor Relacional (Semana 7)	Entregable 2: Motor Multimodal & IA (Semana 16)
Ponderación	Proyecto 1 (P1): 15 %	Proyecto 2 (P2): 15 %
Almacenamiento	Bloques de tamaño fijo (4 KB / 8 KB), Page Layout con cabecera y slots, RID $\langle \text{page_id}, \text{slot} \rangle$, y componente <code>DiskCounter</code> de I/O.	Soporte para colecciones no estructuradas, blobs multimedia y persistencia de índices multidimensionales.
Organización de Archivos	Heap File (con Free-List o Move-the-last) y Sequential File (área principal ordenada, área de overflow encadenada y reorganización periódica).	Integración con almacenamiento de documentos de texto, geometrías cartográficas y vectores de alta dimensión.
Capa de Índices	Árbol B^+ multinivel en disco (split, búsqueda puntual y por rango) y Hashing Dinámico (Extendible Hashing o Linear Hashing).	R-Tree (consultas espaciales/KNN), Índice Invertido SPIMI (BM25/TF-IDF) y Búsqueda Vectorial ANN (IVF o HNSW).
Motor SQL	Soporte SQL relacional: <code>CREATE TABLE</code> , <code>INSERT</code> , <code>SELECT ... WHERE</code> , <code>DELETE</code> y <code>CREATE INDEX</code> .	Sintaxis extendida: distancias geográficas, búsqueda por relevancia textual (<code>MATCH</code>) y similitud de embeddings (<code>COSINE</code>).
Planificador y Métricas	Selección de ruta de acceso (<i>Full Scan</i> vs. <i>Index Scan</i>) con desglose exacto de I/O en disco y latencia en ms.	Optimización híbrida multivariable combinando predicados relacionales con filtros espaciales y umbrales vectoriales.
Frontend Web	Cliente SQL con 4 paneles: explorador de tablas, editor de consultas, visor de resultados y visor gráfico de métricas I/O.	Interfaz completa con visor de mapa interactivo (Leaflet/OpenLayers) y reproductor/visualizador multimedia.
Aplicación IA	No requerida en esta fase (enfoque en cimientos del motor).	Obligatoria: RAG sobre papers, Recomendador e-commerce, Detección de copyright o Reconocimiento facial.
Experimentación	Pruebas de estrés con datasets tabulares $\geq 100k$ registros: Full Scan vs. B^+ vs. Hash (I/O vs. N).	Evaluación de recall vs. latencia en búsqueda vectorial, escalabilidad espacial y tiempo de respuesta extremo a extremo.

3. Entregable 1

3.1. Capa de Almacenamiento Físico

- **Gestión de Bloques/Páginas Físicas:** Todo archivo de datos e índice debe residir en disco organizado en unidades homogéneas de transferencia de tamaño fijo ($B = 4096$ bytes u 8192 bytes). No se permite el uso de serializadores de alto nivel en memoria como `pickle` ni la lectura global de archivos mediante `.read()`. Toda interacción debe realizarse mediante desplazamientos de puntero binario (`seek(offset)`) y empaquetamiento de tipos con `struct` en Python o memoria binaria en C++.
- **Layout de Página (Page Layout):** Cada bloque físico debe incorporar una cabecera (*Page Header*) que contenga:
 - `page_id`: Identificador único del bloque dentro del archivo.
 - `record_count`: Cantidad de registros activos en la página.
 - `free_space_offset`: Puntero al inicio del espacio contiguo libre o cabecera de la lista de slots disponibles.
 - `next_page_id` / `prev_page_id`: Punteros de encadenamiento entre páginas.

Se puede optar por una organización de **Registros de Longitud Fija con Bitmap de presencia** o por la arquitectura estándar de **Página Ranurada (*Slotted-Page Architecture*)**.

- **Identificador de Registro (RID):** Cada tupla se referencia unívocamente mediante la tupla lógica $RID = \langle page_id, slot_number \rangle$.
- **Instrumentación de control (I/O Counter):** El gestor debe implementar de forma obligatoria un monitor (`DiskCounter`) que registre exactamente las variables:

`disk_reads`: bloques físicos leídos, `disk_writes`: bloques físicos escritos

3.2. Métodos de Organización de Archivos

El motor debe ofrecer soporte para dos métodos fundamentales de organización física de tablas:

1. Heap File :

- *Inserción*: Costo $O(1)$ de I/O ubicando la primera página con espacio disponible mediante una lista libre (*Free-List*) o anexando al final del archivo.
- *Búsqueda*: Escaneo secuencial completo (*Full Table Scan*) con costo de P lecturas de disco (donde P es el número total de páginas).
- *Eliminación*: Eliminación lógica con encadenamiento de espacios en la Free-List, o compactación inmediata mediante la técnica *Move-the-last*.

2. Sequential File :

- *Estructura Dual*: Compuesto por un **Área de Datos Principal** (páginas ordenadas físicamente por la clave primaria, admitiendo búsqueda binaria) y un **Área de Desbordamiento (*Overflow Area*)** donde se insertan tuplas que ya no entran en su bloque principal correspondiente, manteniéndose el orden lógico mediante punteros binarios encadenados.
- *Operación de Reorganización (*reorganize*)*: Proceso periódico de mantenimiento que realiza un recorrido ordenado de ambos archivos, fusiona los registros y reescribe un archivo principal limpio y balanceado con un factor de carga (*fill factor*) de 70–80 %, vaciando el área de overflow.

3.3. Capa de Indexación Relacional Avanzada

Se debe implementar en disco dos técnicas de indexación sobre páginas de tamaño fijo:

1. Árbol B^+ multinivel en disco :

- *Nodos Internos*: Almacenan un arreglo ordenado de claves separadoras y punteros a bloques hijos $\langle P_0, K_1, P_1, \dots, K_m, P_m \rangle$.
- *Nodos Hoja*: Almacenan pares ordenados $\langle \text{Key}_i, \text{RID}_i \rangle$ (o el registro completo en índice agrupado) y punteros de enlace continuo `next_leaf_id` y `prev_leaf_id`.
- *Operaciones*: Inserción con **división recursiva (*split*)** al 50 % con propagación hacia el nodo padre y creación de nueva raíz ante sobreflujo; búsqueda puntual con costo acotado a la altura $h = O(\log_M N)$ transferencias de disco; y búsqueda por rango eficiente descendiendo a la primera hoja y recorriendo los punteros horizontales `next_leaf`.

2. Hashing Dinámico en Disco:

- Implementar **Extendible Hashing** o **Linear Hashing**.

3.4. Motor de Consultas y Parser SQL

El sistema debe proveer una interfaz declarativa capaz de interpretar y ejecutar sentencias SQL estructuradas:

```
-- Creacion de tablas con motor de almacenamiento especifico
CREATE TABLE empleados (id INT PRIMARY KEY, nombre CHAR(30), dept CHAR(20)
, salario FLOAT) USING [HEAP | SEQUENTIAL];

-- Insercion y creacion de indices en disco
INSERT INTO empleados VALUES (101, 'Ada Lovelace', 'Analytics', 5200.0);
CREATE INDEX idx_emp_id ON empleados(id) USING [BTREE | HASH];

-- Consultas de seleccion puntual y por rango
SELECT * FROM empleados WHERE id = 101;
```

```
SELECT * FROM empleados WHERE id >= 100 AND id <= 500;
```

Planificador y Telemetría del Plan de Ejecución: El motor debe inspeccionar la cláusula WHERE y seleccionar la ruta óptima:

- Si existe un índice BTREE o HASH aplicable a la igualdad, despachar un **IndexScan**.
- Si se evalúa un rango y existe un índice BTREE, despachar un **IndexRangeScan**.
- En cualquier otro caso, despachar un escaneo secuencial (**SeqScan**).
- La ejecución debe reportar el desglose exacto: bloques leídos, bloques escritos, tiempo de parseo y tiempo de ejecución en milisegundos (ms).

3.5. Backend API REST y Frontend Web (Cliente SQL)

- **Backend (FastAPI / Flask / C++):** Exponer endpoints REST como POST /api/query (procesa la consulta y retorna tuplas más métricas de I/O y tiempo), GET /api/tables y POST /api/tables/reorganize.
- **Frontend Web Interactivo:** Diseñado similar al presentado en la primera sesión teniendo 4 secciones visibles:
 1. *Explorador de Tablas:* Visualización de tablas y tipos de índices activos.
 2. *Editor SQL:* Entrada de consultas con resaltado y botón de ejecución.
 3. *Visor de Resultados:* Tabla paginada con los registros recuperados.
 4. *Plan de Ejecución y Métricas:* Panel con el número exacto de transferencias de disco (I/O Reads / Writes) y latencia en milisegundos.

3.6. Conjuntos de Datos de Prueba

Las pruebas deben realizarse obligatoriamente sobre volúmenes masivos representativos:

- **Volumen Mínimo:** Datasets reales o sintéticos de al menos **100,000 a 500,000 registros** (por ejemplo, transacciones comerciales, registros de empleados o datos de transporte).
- No se admitirán proyectos validados únicamente con muestras triviales de 10 a 50 filas.

4. Experimentación y Rendimiento

El informe técnico debe incluir obligatoriamente los siguientes cuatro experimentos con datos tabulados y gráficas comparativas analizadas a la luz de la teoría:

1. **Experimento 1: Costo de Inserción Masiva:** Comparar el tiempo total y escrituras I/O requeridas para insertar lotes crecientes de tuplas $N \in [10^3, 10^4, 5 \times 10^4, 10^5, 2,5 \times 10^5, 5 \times 10^5]$ en Heap File, Sequential File (con y sin reorganización), Árbol B^+ y Hash Dinámico.

2. **Experimento 2: Búsquedas Puntuales de Igualdad:** Ejecutar un conjunto de 1000 consultas aleatorias de igualdad sobre $N = 100,000$ registros. Comparar el promedio y desviación estándar de I/O reads y latencia (ms) entre:

Full Scan (Heap) vs. Búsqueda Binaria (Sequential) vs. Árbol B^+ vs. Hash Dinámico

3. **Experimento 3: Búsquedas por Rango con Selectividad Variable:** Evaluar el costo de I/O y tiempo al consultar rangos que representan el 0,1 %, 1 %, 5 %, 10 % y 25 % del total de tuplas. Contrastar el Árbol B^+ frente al Sequential File y al Full Scan.
4. **Experimento 4: Sensibilidad al Tamaño de Bloque:** Variar el tamaño de página $B \in [1024, 2048, 4096, 8192]$ bytes y analizar el efecto en el factor de ramificación (*fan-out*), la altura h del árbol B^+ y el número total de I/Os transferidos.

5. Entregable 2

En el Entregable 2, el motor relacional construido se expandirá para soportar datos no convencionales y una aplicación con Inteligencia Artificial:

- **Parte 2 - Módulo Espacial (R-Tree / GiST):** Indexación 2D en disco basada en cajas envolventes mínimas (MBR), consultas de proximidad y visualización en mapa cartográfico integrado.
- **Parte 3 - Módulo de Texto:** Construcción externa de índices invertidos mediante el algoritmo SPIMI y ranking de relevancia por contenido mediante **TF-IDF** y **BM25**.
- **Parte 4 - Módulo Multimedia y Vectorial:** Indexación de representaciones densas de alta dimensión (*embeddings*) mediante **IVF** o grafos navegables **HNSW**.
- **Parte 5 - Aplicación de IA :** Despliegue de una de las 4 opciones propuestas:
 - *Opción A:* Sistema RAG (*Retrieval-Augmented Generation*) sobre literatura científica (ArXiv / NeurIPS).
 - *Opción B:* Sistema Recomendador multimodal para comercio electrónico (imágenes similares + metadatos).
 - *Opción C:* Motor de detección de huellas acústicas y derechos de autor en pistas de audio.
 - *Opción D:* Sistema biométrico de reconocimiento e identificación facial (LFW / CelebA).

6. Instrucciones de Entrega (Semana 7)

1. **Repositorio de Código (GitHub):** Repositorio con acceso otorgado al docente conteniendo: carpetas `backend/`, `frontend/`, `data/`, `benchmarks/` y un archivo `README.md` con instrucciones precisas para levantar el sistema en un solo paso.

2. **Informe:** Documento en formato PDF compilado desde L^AT_EX conteniendo la descripción del diseño físico, decisiones de arquitectura y el reporte experimental detallado.
3. **Video Demostrativo (5 a 10 minutos):** Grabación donde el equipo inspeccione los archivos binarios en disco, demuestre la ejecución de consultas en el cliente web observando el plan de ejecución y presente los resultados del benchmark automatizado.